Solid principles

Sunday, 4 January 2026 9:43 AM

LLD :-

↳ Low Level Design

readable & maintainable
extensibility
reuse

↳ To enhance code & developer efficiency

SOLID

↳ guidelines
↳ 2000
↳ easy to maintain & extend

S → Single Responsibility Principle
O → Open / closed principle
L → Liskov Substitution
I → Interface segregation
D → Dependency Inversion

S → Single Responsibility Principle

A class should have one & only one reason to change.
Hence, a class should only have one job.

Open-Closed principle

Classes should be open for extension & closed for modification.

Liskov- substitution principle

Let $q(x)$ be a property provable for x of Type T. Then $q(y)$ should be provable for objects y of type S which is a subtype of T.

$\Rightarrow$ Every subclass must be substitutable for their base or parent class.

I → Interface Segregation

Client should never be forced to implement an interface that it doesn't use, or clients shouldn't be forced to depend on methods they do not use.

D → Dependency Inversion DI

Entities must depend on abstractions not on concretions. It states that the high level module must not depend on the low level module but they should depend on abstractions.

Area Calculator

Area Printer

th

** Refer to github repo for the code